BENEMÉRITA UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Ciencias Computacionales

Regresión Lineal Simple

2° Actividad

Aprendizaje Máquina

Código de la materia: IL381

Sección: D02

P R E S E N T A(N)

ICOM - Barrera Hermosillo Elizabeth Kristie - 220916338

ICOM -Rodríguez Luna Paulina - 223378663

ICOM - Velázquez Vazquéz Valeria - 224785718

PROFESOR(A)

María Fernanda López Aramburo

Guadalajara, Jalisco, Mx.

3 de Septiembre del 2026

ÍNDICE

Índice 2

Actividad: Regresión Lineal Simple 3

Instrucciones 3

Entregables .....

4 Durante la clase .....

4

1. Resolución Manual 5

1. planteamiento del problema 5

2. Tabla de Datos y Cálculos Intermedios 6

3. Sustitución en las fórmulas 6

4. Ecuación de la Recta de Regresión 8

5. Predicción Manual 9

2. Programación en Python 10

3. Visualización de Resultados 12

4. Resultados en Tabla y Valores 13

Comparación entre los valores manuales y programados 14

Conclusiones: Codificar vs calcular a mano 14

INSTRUCCIONES

EN CASA

Resolver y programar el **Ejemplo 1 de Regresión Lineal Simple**, siguiendo los pasos que se indican a continuación.

1. RESOLUCIÓN MANUAL

- Calcular la **pendiente** y la **intersección** de la recta.
- Realizar una **predicción de manera manual**, mostrando todos los cálculos realizados.

2. PROGRAMACIÓN EN PYTHON CON SCIKIT-LEARN

- Implementar el modelo utilizando `LinearRegression()` de **Scikit-learn**.
- Dividir el conjunto de datos en: ○ **80 % para entrenamiento**.
  - ○ **20 % para prueba**.
- Comparar los resultados obtenidos mediante el cálculo manual con los resultados del modelo programado.
- Comentar en el código cada uno de los pasos realizados.

3. VISUALIZACIÓN DE RESULTADOS

Generar las siguientes gráficas:

- **Gráfica 1:** Mostrar los datos iniciales, diferenciando mediante colores los conjuntos de **entrenamiento** y **prueba**.
- **Gráfica 2:** Mostrar los datos junto con la **recta ajustada por el modelo**.

4. RESULTADOS EN TABLA Y VALORES

Generar un **DataFrame** que contenga las siguientes columnas:

Columna
Experiencia
Salario real
Salario predicho

Además:

- Mostrar el valor final de la **pendiente** obtenida por el modelo.
- Mostrar el valor final de la **intersección** obtenida por el modelo.

ENTREGABLES

Se deberán entregar los siguientes archivos:

- **Código:** Archivo con extensión `.py` que contenga el programa desarrollado.
- **Cálculo manual:** Archivo en formato **PDF** con el procedimiento y todos los cálculos realizados.

DURANTE LA CLASE

Durante la clase, se seleccionará **aleatoriamente a un equipo** para que comparta con el grupo:

- El **código desarrollado**.
- Los **resultados obtenidos**.
- La **comparación entre los valores calculados manualmente y los obtenidos mediante Python**.
- Las **conclusiones** sobre la experiencia de:
  - Codificar el modelo. ○ Realizar los cálculos manualmente.
  - Comparar ambos métodos.

1. RESOLUCIÓN MANUAL

1. PLANTEAMIENTO DEL PROBLEMA

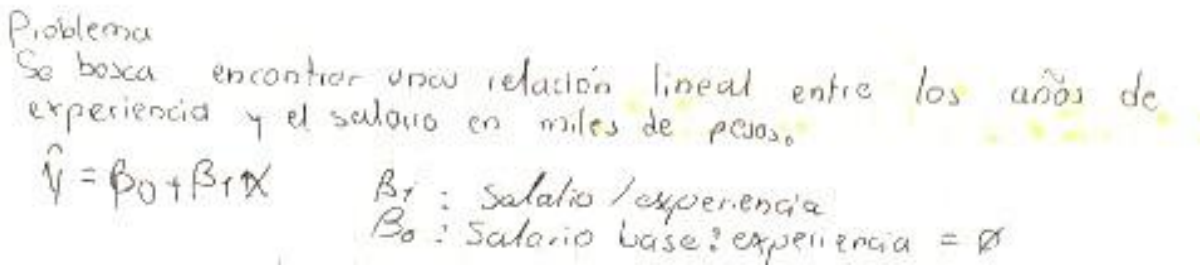
Se busca encontrar una relación lineal entre los años de experiencia (X) y el salario real en miles de MXN (Y). El modelo matemático de la regresión lineal simple es:

$$\hat{Y} = \beta_0 + \beta_1 X$$

Donde:

• β_1 (pendiente): Indica cómo cambia el salario por cada año adicional de experiencia.

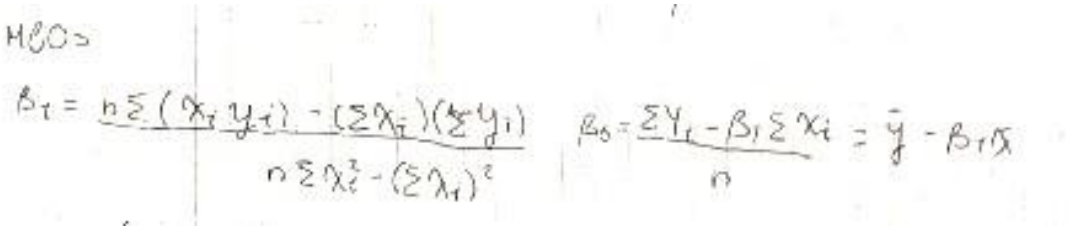
• β_0 (intersección): Es el salario base esperado cuando los años de experiencia son 0.



Las fórmulas para encontrar los parámetros por el método de Mínimos Cuadrados Ordinarios (MCO) son:

$$\beta_1 = \frac{n\sum(X_iY_i) - (\sum X_i)(\sum Y_i)}{n\sum X_i^2 - (\sum X_i)^2}$$

$$\beta_0 = \frac{\sum Y_i - \beta_1 \sum X_i}{n} = \bar{Y} - \beta_1 \bar{X}$$



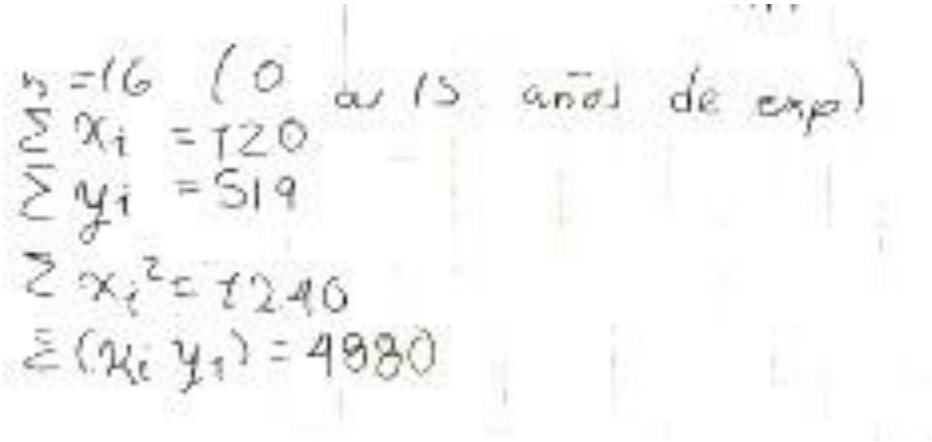
2. TABLA DE DATOS Y CÁLCULOS INTERMEDIOS

Aunque el problema indica datos de 15 personas, la tabla mostrada contiene 16 registros (de 0 a 15 años de experiencia). Por lo tanto, $n = 16$. A continuación se calcula cada componente necesario:

3. SUSTITUCIÓN EN LAS FÓRMULAS

Con la tabla anterior, obtuvimos los siguientes valores totales:

$$n = 16$$
$$\sum X_i = 120$$
$$\sum Y_i = 519$$
$$\sum X_i^2 = 1240$$
$$\sum (X_iY_i) = 4880$$



Dato (i)	Experiencia (X_i)	Salario (Y_i)	X_i^2	X_iY_i
1	0	12	0	0
2	1	14	1	14
3	2	17	4	34
4	3	19	9	57
5	4	22	16	88
6	5	25	25	125
7	6	27	36	162
8	7	30	49	210
9	8	32	64	256
10	9	35	81	315
11	10	42	100	420
12	11	44	121	484
13	12	46	144	552
14	13	48	169	624
15	14	51	196	714
16	15	55	225	825
Suma ($\sum$)	**120**	**519**	**1240**	**4880**

CÁLCULO DE LA PENDIENTE (B1)

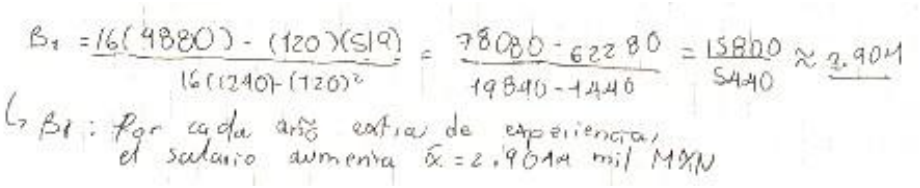
Sustituyendo los valores en la fórmula de la pendiente:

$$\beta_1 = \frac{16(4880) - (120)(519)}{16(1240) - (120)^2}$$

$$\beta_1 = \frac{78080 - 62280}{19840 - 14400}$$

$$\beta_1 = \frac{15800}{5440} \approx 2.9044$$

Interpretación: Por cada año adicional de experiencia, el salario aumenta en promedio 2.9044 mil MXN.



CÁLCULO DE LA INTERSECCIÓN (B_0)

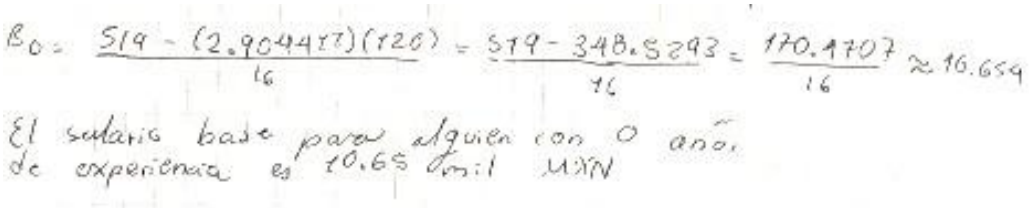
Sustituyendo el valor de β_1 y las sumatorias en la fórmula de la intersección:

$$\beta_0 = \frac{519 - (2.904411)(120)}{16}$$

$$\beta_0 = \frac{519 - 348.5293}{16}$$

$$\beta_0 = \frac{170.4707}{16} \approx 10.6544$$

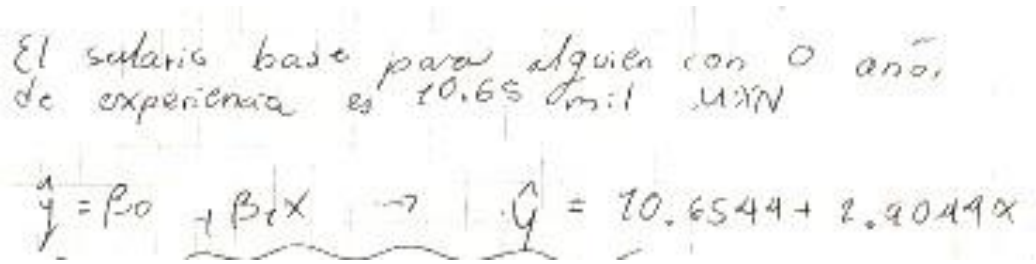
Interpretación: El salario base estimado (teórico) para alguien con 0 años de experiencia es 10.6544 mil MXN.



4. ECUACIÓN DE LA RECTA DE REGRESIÓN

Sustituyendo los parámetros en el modelo, la ecuación de regresión ajustada es:

$$\hat{Y} = 10.6544 + 2.9044X$$

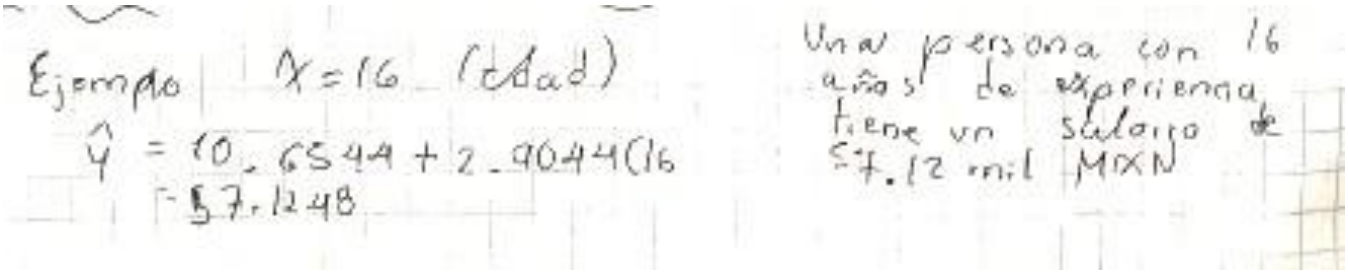


5. PREDICCIÓN MANUAL

Realicemos una predicción hipotética para una persona con 16 años de experiencia (X = 16), mostrando todos los cálculos:

$$\hat{Y} = 10.6544 + 2.9044(16)$$
$$\hat{Y} = 10.6544 + 46.4704$$
$$\hat{Y} = 57.1248$$

Conclusión de la predicción: Según nuestro modelo ajustado a mano, se espera que una persona con 16 años de experiencia laboral tenga un salario aproximado de 57.12 mil MXN.



```
# Importar librerías import numpy as np # numpy: manejo de arreglos numéricos
import pandas as pd # pandas: tablas de datos
import matplotlib.pyplot as plt # matplotlib: visualización from sklearn.linear_model import LinearRegression
#scikit-learn: modelo de
regresión lineal
from sklearn.model_selection import train_test_split #scikit-learn: ayudante para distribuir datos
entre entrenamiento y prueba
# Inicializacion de los arreglos que contienen los datos a tratar, y es el salario y X son los años de experiencia
y = np.array([12,14,17,19,22,25,27,30,32,35,42,44,46,48,51,55]) X = np.array([[i] for i in range(0,
len(y))]) # X es un arreglo bidimensional de tamaño (len(y), 1) x = X.flatten() # Variable ayudante
cuando es necesario iterar por X,
requiriendo un arreglo unidimensional

X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8) #
Distribución de los datos entre entrenamiento y prueba

model = LinearRegression().fit(X_train, y_train) # Entrenamiento del modelo
 score = model.score(X_test, y_test) # Coeficiente de determinación, que tan
bueno es el modelo para predecir los valores de prueba basado en su entrenamiento

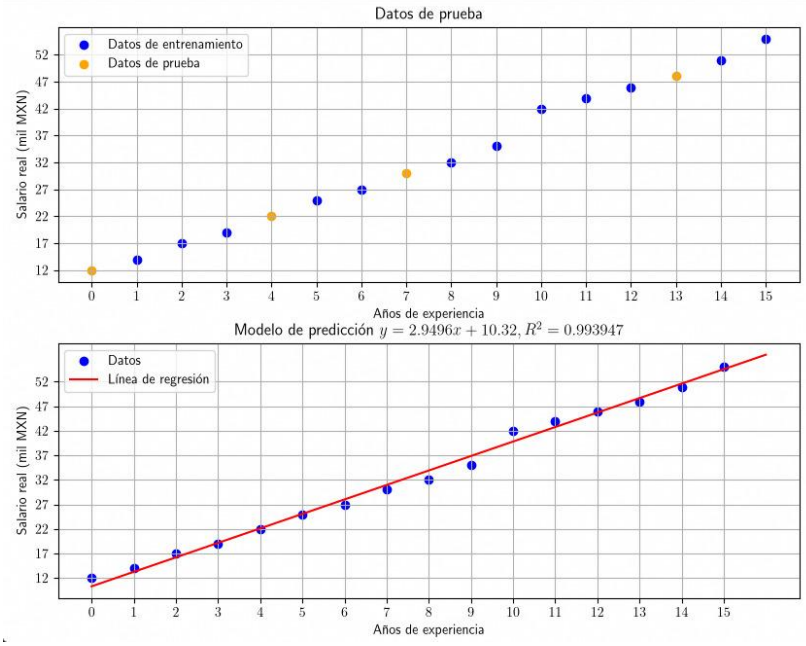
alpha = model.coef_[0] #Pendiente de la recta modelada
beta = model.intercept_ #Punto de intercepción de la recta modelada

fig = plt.figure(figsize=(14, 8)) #Inicialización de la figura de
matplotlib
ROWS, COLS = 2,5
COLS_FOR_PLOTS = 3
 plt.rcParams['text.usetex'] = True #Habilita el renderizado de ecuaciones
con LaTeX
# Renderizado de la primera gráfica, que muestra la distribucion entre datos de entrenamiento y de
prueba ax1 = plt.subplot2grid((ROWS, COLS),(0,0), colspan=COLS_FOR_PLOTS) ax1.set_title("Datos de
prueba")
ax1.scatter(X_train, y_train, color="blue", label="Datos de entrenamiento") ax1.scatter(X_test, y_test,
color="orange", label="Datos de prueba")
```

```
# Renderizado de la segunda gráfica, que muestra la línea de regresión contra los datos iniciales
ax2 = plt.subplot2grid((ROWS, COLS),(1,0), colspan=COLS_FOR_PLOTS) ax2.set_title(f"Modelo de predicción $y
= {np.round(alpha, decimals=4)}x +
{np.round(beta, decimals=4)}, R^2={np.round(score, decimals=6)}$") #Muestra el titulo con el valor de la
recta modelada y el coeficiente de determinacion
ax2.scatter(X, y, color="blue", label="Datos") x_predict = np.linspace(0,
len(y), 100) y_predict = x_predict * alpha + beta
ax2.plot(x_predict, y_predict, color="red",label="Línea de regresión")
# Configurar todos los graficos de la misma forma for ax in [ax1, ax2]:    ax.set(xlabel="Años de experiencia",
ylabel="Salario    real    (mil    MXN)")                    ax.xaxis.set_ticks(np.arange(min(x),max(x)+1,    1.0))
ax.yaxis.set_ticks(np.arange(min(y),max(y)+1, 5.0))    ax.legend()    ax.grid()
# Creacion del DataFrame para comparar directamente datos reales contra predicciones
df = pd.DataFrame() df['Experiencia']=x
df['Salario real']=y
df['Salario predicho']=np.round(X*alpha + beta, decimals=2)
# Renderizado de la tabla junto a las otras dos figuras ax3 = plt.subplot2grid((ROWS, COLS),
(0,COLS_FOR_PLOTS), rowspan=2, colspan=COLS - COLS_FOR_PLOTS)
ax3.axis("off")
table = ax3.table(cellText = df.values, rowLabels = df.index, bbox=[0,0,1,1],
colLabels=df.columns) table.auto_set_font_size(False) table.set_fontsize(12)

# Mostrar la figura plt.tight_layout()
plt.show()
```

3. VISUALIZACIÓN DE RESULTADOS



	Experiencia	Salario real	Salario predicho
0	0.0	12.0	10.32
1	1.0	14.0	13.27
2	2.0	17.0	16.22
3	3.0	19.0	19.17
4	4.0	22.0	22.12
5	5.0	25.0	25.07
6	6.0	27.0	28.02
7	7.0	30.0	30.97
8	8.0	32.0	33.92
9	9.0	35.0	36.87
10	10.0	42.0	39.82
11	11.0	44.0	42.77
12	12.0	46.0	45.71
13	13.0	48.0	48.66
14	14.0	51.0	51.61
15	15.0	55.0	54.56

- Gráfico superior izquierdo (División de datos): Se muestra la relación entre los años de experiencia y el salario real. Los puntos azules corresponden al conjunto de entrenamiento y los naranjas al conjunto de prueba.

- Gráfico inferior izquierdo (Línea de regresión): Se muestran todos los datos reales (puntos azules) atravesados por la recta de predicción (línea roja). Con esta línea se representa visualmente la ecuación del modelo y se ilustra la tendencia central de cómo aumenta el salario con los años.

- Tabla de la derecha (Comparativa numérica): Se expone el desglose exacto de los resultados. Se compara numéricamente el salario real histórico frente al salario predicho para cada nivel de experiencia, del año 0 al 15.

4. RESULTADOS EN TABLA Y VALORES

```
# Creacion del DataFrame para comparar directamente datos reales contra predicciones df = pd.DataFrame() df['Experiencia']=x df['Salario real']=y
df['Salario predicho']=np.round(X*alpha + beta, decimals=2)
```

	Experiencia	Salario real	Salario predicho
0	0.0	12.0	10.32
1	1.0	14.0	13.27
2	2.0	17.0	16.22
3	3.0	19.0	19.17
4	4.0	22.0	22.12
5	5.0	25.0	25.07
6	6.0	27.0	28.02
7	7.0	30.0	30.97
8	8.0	32.0	33.92
9	9.0	35.0	36.87
10	10.0	42.0	39.82
11	11.0	44.0	42.77
12	12.0	46.0	45.71
13	13.0	48.0	48.66
14	14.0	51.0	51.61
15	15.0	55.0	54.56

COMPARACIÓN ENTRE LOS VALORES MANUALES Y PROGRAMADOS

- Valores del cálculo manual: La pendiente y la intersección se calcularon tomando el 100% de los datos (los 16 registros).

- Valores del modelo programado: En la ecuación del modelo, se muestran valores ligeramente distintos. Esto ocurre porque los datos se dividieron (80% para entrenamiento y 20% para prueba). La recta se ajustó utilizando los datos de entrenamiento, mientras que el resto para la evaluación.

- Conclusión de la comparación: Ambos resultados son correctos y se concluye que la diferencia no es un error. Simplemente se refleja el uso de distintas cantidades de datos para el cálculo.

CONCLUSIONES: CODIFICAR VS CALCULAR A MANO

- El valor del cálculo manual: Al realizar el proceso a mano, se elimina el efecto de "caja negra" del algoritmo. De esta forma, se comprende matemáticamente cómo se minimiza el error entre los puntos y la recta. Sin embargo, se considera un proceso lento y propenso a errores (si un cálculo se altera por equivocación, todo el resultado se afecta).

- El poder de la programación: Mediante el código se obtiene velocidad, automatización y escalabilidad. En pocas líneas se logra lo que a mano requiere varias páginas de cálculos. Además, con el uso de librerías se facilitan procesos complejos, como la partición de datos y la evaluación del modelo.

- Veredicto final: Se reconoce que el cálculo manual es fundamental para entender la base teórica, pero se concluye que la codificación es la herramienta indispensable con la cual se aplican verdaderamente estos modelos en el mundo real de manera eficiente.